

底層邏輯深挖

核心演算法逐一拆解 — 資料結構 · 控制流 · 門檻常數 · 邊界情況

* 動態資料 (持續新增/更新) · 快照 2026-06-22 · 直讀原始碼非轉述 *

本文與既有技術報告的差別：前者講『做什麼』，本文講『怎麼算』 -- 直接讀
個核心模組的實際程式碼，把真正的演算法、判斷分支、門檻數值、資料形狀與『為什麼這樣寫』的邊界考量挖出來。每節
附原始碼位置，可逐一覆核。

10

涵蓋：(1)DAG 排程 (2)步驟引用解析 (3)成本治理 (4)計費公式 (5)感知迴圈 (6)自省 (7)生成鎖 (8)LLM 容錯/斷路 (9)Migration 三鐵則
(10)JWT 硬化。

1. DAG 排程器 - Kahn 拓撲分批 + 同頁隱式串接 + allSettled 不重跑

原始碼：shared/orb-dag-scheduler.ts

資料結構：incoming/outgoing 各一個 Map<stepId, Set<stepId>>（用 Set 去重--顯式 dependsOn 與隱式同頁邊可能重覆加）。

演算法（Kahn）：(1) 先建顯式 dependsOn 邊（跳過自環與不存在的 id）。(2) 加『同頁隱式邊』--掃 lastSeenByPath，同一個非空 path 的每一步都 depends on 同 path 的前一步（防兩個 UI 派發同頁並發 = 光球頭號競態）。(3) 反覆取出 remaining 入度為 0 的所有步驟當一個 batch、移除其出邊；若某輪取不到任何 ready 但還有未訪節點 = 有環，回傳 cycle（剩餘節點清單），orchestrator fail-safe 退回循序。

關鍵邊界（執行端 runBatchesWithConcurrency）：刻意不用『Promise.all 一失敗整片重跑』--那會把已成功的 step 再跑一次（重複扣點/重複生成/results 重塞）。改成每個 step try/catch 包成 {ok,value}{ok:false,error}，成功塞 results、遇第一個失敗即回報 {results, failed:{step,error}}，不重跑成功項。並發以 cap=max(1,floor(concurrency)) 在 batch 內切片節流。

```
while(visited<steps){ ready = steps.filter(s=>!visited(s)&&remaining(s).size===0); if(ready.length===0) return {cycle: 剩餘節點}; batches.push(ready); 移除 ready 出邊 }
```

2. 步驟引用解析 - \${stepN.key} 執行期替換 + 型別保留 + 不靜默丟

原始碼：shared/orb-step-ref-resolver.ts

問題：多步計畫 step2 需要 step1 的非同步產物（如 video_url），planner 無法事先猜值。解法：planner 寫 toolArgs={video_url:"\${step1.video_url}"}, 執行期由 resolver 對『累積結果』替換真值。

規則：(1) 正則 /\\$\{([^\}]+)\}/g 抓佔位；getByPath 逐層走 toolResult.data[key]...（可選 steps. 前綴、output. 別名）。(2) 若整個字串就是單一 \${...} 且解析值非字串 -> 保留原型別（number/boolean/array），不被強轉成字串。(3) 佔位混在其他文字中 -> best-effort 轉字串拼接（用於 prompt）。(4) 解不到的佔位『留在原地』不靜默丟 -> orchestrator 偵測得到缺漏並大聲失敗，而不是把 undefined 偷偷送進工具。

3. 成本治理 - 保守成本表 + maxTier + workflow 估算

原始碼：shared/orb-cost-governor.ts

CostEntry={credits, durationMs, tier}. tier 五級 free/cheap/medium/expensive/premium, TIER_RANK 0..4 取 maxTier。成本表（刻意略高估）：

工具/動作	credits	時長	tier
studio.generateImage	8 cr	12s	cheap
studio.generateVideo	60 cr	90s	expensive
studio.animateSpeaker	80 cr	120s	premium
studio.trainLora	200 cr	20 min	premium
UI submit	5 cr	15s	cheap
UI 其他(fillPrompt/setModel..)	0 cr	1.5s	free
未知工具	10 cr(保守)	30s	medium

estimateWorkflowCost 累加每步 credits/durationMs、取 maxTier、標 hasBillableStep。未知工具給保守 medium 佔位，讓門檻在 registry 長新工具比成本表快時仍可預期。此估算純前端、不洩定價細節到 client（真實逐模型價在 modelPricing）。

4. 計費公式 estimatePoints - 加法 baseline + 防雙倍計費

原始碼：server/services/modelPricing.ts

公式（加法，未知模型固定 5 pts）：

```
total = basePoints + round( max(0, durationSec - freeSecondsInBase) x pointsPerSecond ) + ceil( charCount/1000 x pointsPer1kChars ) + max(0, imageCount-1) x pointsPerImage + trainingSteps x pointsPerStep , 再 clamp 到 [minPoints, maxPoints]
```

防雙倍關鍵：basePoints 已內含 freeSecondsInBase 秒的 baseline（例 Kling V2.1 Std basePoints=30 對應 5s）。時長加收只算『超過 baseline 的秒數』（durationSec-freeSec），否則同 5 秒會被 basePoints 與 pointsPerSecond 各收一次。asPositiveNumber 把非數值/負數歸 0（防呆）；每段產生人類可讀 breakdown 字串供稽核。1 USD = 100 pts。

5. 感知迴圈 - 快照逐欄 diff + 動作預期 + 保守判定

原始碼：shared/orb-perception-loop.ts

SnapshotDelta 逐欄布林：pageChanged/modelChanged/modeChanged/modalityChanged/promptChanged/presetChanged，外加 newWarnings/resolvedWarnings/changedStateKeys/anyChange。diffSnapshots 對兩張 PageAgentSnapshot 逐欄比（valueKey 用 JSON.stringify 正規化）。

expectedDeltaForAction：每種動作對應一個『該翻 true 的欄位』--navigate->pageChanged、setModel->modelChanged、fillPrompt->promptChanged、applyPreset->presetChanged...。刻意把 submit/focusElement/runWorkflow 設 expectsChange=false（submit 啟動非同步生成、focus 純視覺、巢狀工作流逐步另判），避免把『本來就不會立刻變』誤判成 no-effect。

判定保守原則（原碼註解）：『寧可少報成功，也不把 no-effect 的步驟宣告成功』。outcome ∈ succeeded/no-effect/diverged/unknown -> recommendation ∈ proceed/retry/replan/abort。快照任一張缺失 -> 零 delta，交由 verdict 層決定算 unknown 或『無觀測』，而非當成成功。

6. 自省 - 門檻 + severity 升級（永不拋例外）

原始碼：shared/orb-self-reflection.ts

常數：SLOW_LATENCY_MS=60000、MAX_HEALTHY_RETRIES=1。severity 三級 info<warn<fail。檢查項：

(1) 空/過小輸出 -> warn (code reflection:empty-*) (2) 重複同款失敗簽名 -> warn/fail (3) latencyMs>60s 的成功 -> warn (slow-latency，結果可用但偏慢)。升級規則 shouldEscalate：severity===fail 一律升級；warn 且 code 以 reflection:empty-開頭也升級（空輸出視同需要 replan）。永不對畸形輸入拋例外--回 severity=info，確保壞掉的自省不會拖垮真實工具執行。verdict 寫回 orbMemory 供下輪 planner 學習。

7. 生成防重複鎖 - SET NX PX + Lua CAS-del 自己 token + 200ms fail-open + 晚到自癒

原始碼：server/_core/generationLock.ts / redisGenerationLockStore.ts

語義：acquire = SET key token NX PX ttl (不存在才設、帶 TTL)，成功回 token、被占回 null。release = Lua `if redis.call('get',k)==token then return redis.call('del',k) end` (CAS-del) --保證只刪『自己那把』鎖，TTL 到期後另一請求拿到同 key 也絕不會被你誤刪。

可用性鐵則：所有 store 操作包 200ms 超時 (STORE_OP_TIMEOUT_MS) + try/catch；超時或錯誤一律當『鎖不可用』-> fail-open (放行生成，絕不擋使用者)。這是『便利鎖、非安全控制』的刻意取捨。

晚到自癒 (Redis 邊界)：慢的 setNxPx 可能在 200ms 超時、我們已 fail-open 之後才落地 Redis，留下一把沒人追蹤的鎖 (會 TTL 內誤擋後續合法重試)。故每次 fail-open acquire 後，排程對『自己的 token』做有界、間隔的 best-effort CAS-del，把晚到的鎖補刪掉。未設 REDIS_URL -> 退記憶體版 (單機正確)；多 replica 才需 Redis。

8. LLM 容錯 - 錯誤分級 + 斷路器 (3 連敗/5 分) + 不誤斷路

原始碼：server/jobs/circuitBreaker.ts / LLM invoke

錯誤分級 classifyLLMError：transient (暫時，可重試) / permanent_model (4xx invalid_model，跳過重試立即回退) / quota / auth。permanent_model 排在 quota/auth 之後 (不遮蔽 402/401/403)，且用獨立 modelInvalidCount 觀測計數、不開斷路器--避免把健康引擎因『模型名打錯』而誤熔斷 (AIDV-165 真實事故)。

斷路器 CircuitBreaker：連續失敗達門檻 (預設 3) -> OPEN (跳過呼叫)；冷卻 (預設 5 分) 後 -> HALF_OPEN 試探；成功 -> CLOSED。用於 news/Replicate/learn 等外部 API，避免對掛掉的供應商狂打。配合 auto 路由優先序 (OpenRouter>Anthropic>Gemini>NVIDIA>Vertex>Forge) 達成 fail-down 不 fail-stop。

9. Migration 可重跑三鐵則 (守門測試擋回歸)

原始碼：server/migration-prod-pending-block.test.ts

背景：0067 曾用 MySQL 不支援語法 + 多語句未分段 -> 失敗回滾、0066 裸 ALTER 已生效不回滾 -> 每次開機撞『欄位已存在』卡死整個 runner。事故制度化成三條鐵則 (對 0063 以後『生產可能整段重跑』的 PENDING_BLOCK 條目)：

(1) 禁用 MySQL 不支援的 CREATE INDEX IF NOT EXISTS (全 drizzle/ 適用)。(2) 每個 -> statement-breakpoint chunk 只能一個語句 (mysql2 migrator 逐 chunk 執行、未開 multipleStatements)。(3) ALTER TABLE / CREATE INDEX 必須有 information_schema 守門 (前檢存在才執行，裸寫重跑必爆)。守門測試把三鐵則寫死，任何違反即紅燈擋下。

10. JWT 硬化 - fail-fast 不靜默用弱密鑰

原始碼：server/_core/googleAuth.ts / env.validated.ts

MIN_JWT_SECRET_LENGTH=16。讀取順序 process.env.JWT_SECRET (AUTH_SECRET 別名於 env.validated 已映射) -> ENV.cookieSecret。正式環境 (NODE_ENV==='production') 若密鑰缺失/空白/<16 字元 -> 開機直接 throw (fail-fast)，絕不靜默用弱/空密鑰簽 token。trim() 正規化集中在開機 selfRepairEnv，故下游自始是 trim 後單一真值。Access token 壽命由 365 天縮到 30 天 (AIDV-59)。

貫穿這 10 個機制的共同設計哲學

[*] fail-open vs fail-fast 因地制宜：便利型 (生成鎖) 寧可放行不擋人；安全型 (JWT) 寧可開機就爆不冒險。

[*] 防『重複付費』是一級議題：DAG 不重跑成功步、生成鎖 CAS 只刪自己、計費 freeSecondsInBase 防雙倍、idempotency 唯一鍵。

[*] 保守判定優於樂觀：感知迴圈寧可少報成功；成本表略高估；未知工具給保守 medium。

[*] 邊界情況寫進程式：晚到鎖自癒、解不到的引用大聲失敗、畸形輸入回 info 不拋、環偵測退循序。

[*] 事故制度化成守門測試：migration 三鐵則、PENDING_BLOCK，把『卡死生產的 PO 教訓』變成不可回歸的紅線。